

Stage 1, Machine Learning project

Problem formulation

The aim of this project is to find the expected rating gain from contestants in competitive programming contests. The website used to find data, Codeforces, works with an elo rating similar to chess, and each contest affects this rating. The goal is to determine the elo gain from this contest given the past performances of an individual. It's most certainly impossible to reach a perfect score in that kind of task, but I will try to make it work as much as possible. This is a supervised regression problem: the label is `rating_change = new_rating - old_rating`, i.e. how much elo a contestant gains or loses from that one contest. The dataset consists of 11,917,755 datapoints that I collected, all representing a performance of some individual in a given contest. There are 974,594 accounts in total, and 1796 contests, ranging from 2010-02-19 to 2026-09-13.

The data is made of a lot of flags, showing for instance in which division is the contestant competing, and a lot of continuous data, like the current elo of the individual, etc. This will be detailed more later, but there are 18 features, among which 9 continuous ones, 8 binary ones, and an integer.

Methods

I first of all scrapped every contest I could find on codeforces.com, through their API. There were 2,145 of them, but 349 had no rated participants, thus I just removed them and had 1,796 contests left. The API gave 7 out of the 18 features that are included, and the rest of them were constructed to make more sense of the data. This was especially important because I was planning on using Linear Regression as the first method, and I knew that adding new features, which give some non-linearity, was going to be useful to get better results.

What also matters a lot in determining the next performance in an upcoming contests is how many contests the individual took part in, because codeforces gives a serious boost to the first few contests. Accounts created since 2020 have a mean of +400, +270, +190, +100, +65, +25 of elo gain over the first 6 contests.

Feature selection

It used to work the other way before 2020 though: accounts started at 1500 elo directly and actually tended to lose some on their first contest instead (around -60 on average). This convinced me that whether an account is at its very first contest matters a lot on its own, on top of just how many contests it played before, so I made a binary flag for that (`is_debut`) and kept it alongside the contest count. For the 8% of rows that are an actual debut, all the “past contest” features obviously don't exist yet, so I just set them to 0, which combined with the flag

doesn't bias anything since multiplying by 0 cancels out whatever the model would have done with that feature.

To check that the features I picked actually make sense and not just guesses, I looked at how each one correlates with the elo change on the training rows. The strongest ones are the elo relative to the field average (-0.58), the debut flag ($+0.47$) and the log of the past contest count (-0.46). Some barely correlate at all, like the number of days since the last contest (-0.04) or the average past rank (-0.03), but I kept them anyway since correlation only looks at one feature at a time and won't catch interactions between features.

feature	corr.	feature	corr.
rating_vs_field	-0.58	prior_best_rank	$+0.12$
is_debut	$+0.47$	division_Div4	$+0.13$
log1p_prior_contest_count	-0.46	division_Div3	$+0.10$
prior_avg_rating_change	$+0.37$	division_Div2	-0.10
prior_rating_change_std	-0.34	division_Div1	-0.05
field_avg_rating	-0.27	days_since_last_contest	-0.04
rating_trend_last3	$+0.29$	prior_avg_rank	-0.03
log_num_participants	$+0.22$	division_Global	-0.01
		division_Div1+2	-0.01
		division_ICPC	-0.01

Something I made sure NOT to use as a feature is the rank obtained in the contest itself, or the new rating. Codeforces computes the elo change almost directly from the rank, so giving that to the model would basically be handing it the answer instead of making it predict anything.

Model

I went with Linear Regression to start with. It predicts the elo change as a linear function of the 18 features:

$$h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = \sum_{j=1}^{18} w_j x_j + b,$$

where $\mathbf{x}$ is a data point's feature vector and $\mathbf{w}, b$ are learned from the training data. One of those 18 features is the elo relative to the field, $x_{\text{field}} = \text{old_rating} - \text{field_avg_rating}$. When I plotted the elo change against it, the relationship looked roughly linear (slope around -0.18), the weights stay easy to read afterwards, and fitting it is basically instant even on close to 10 million rows since it has a closed-form solution. I know it can't really capture the non-linear boost new accounts get though, so that's a limitation I'm aware of for this first method.

Loss function

I trained it by minimizing the squared error, averaged over the n training points:

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n (y_i - h(\mathbf{x}_i))^2.$$

It's the standard loss for a regression problem like this one, and it has a closed-form solution, the normal equations, so there's no need to iterate:

$$\mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{y}.$$

To judge how good it is I look at the mean absolute error, directly in elo points, and the root mean squared error.

Validation

For splitting the data I didn't do it randomly. Codeforces changed its rating system over time (see the 2020 change above) and the whole population of players evolved a lot too, so a random split would basically let the model peek at the future while training. Instead I sorted everything chronologically and cut it into 80% training, 10% validation and 10% test, making sure each cut falls between two contests and never in the middle of one.

set	rows	share	period
training	9,536,412	80.0%	2010-02-19 to 2025-01-12
validation	1,189,681	10.0%	2025-01-17 to 2025-10-10
test	1,191,662	10.0%	2025-10-12 to 2026-09-13

Right now the linear regression gets a validation MAE of 54.27, against 101.88 for just predicting the average elo gain every time, so it's already clearly doing something, even if there's obviously a hard limit to how well this can ever work, since a lot of what decides a performance just can't be known ahead of time.

	MAE	RMSE
training	56.38	73.38
validation	54.27	72.76
baseline (predict training mean)	101.88	147.06

Use of AI

I used Claude to typeset this document, and also to debug a lot of the functions that I was using, whether it is for the scraping, the data processing, or the training. I wrote this document myself and asked Claude to correct mistakes.

References

1. Codeforces. <https://codeforces.com>
2. Codeforces API documentation. <https://codeforces.com/apiHelp>

Appendix: code

scrape.py

```
1 import json
2 import time
3 from pathlib import Path
```

```

4
5 import requests
6
7 API = "https://codeforces.com/api"
8 OUT = Path(__file__).resolve().parent.parent / "data" / "rating_changes"
9 OUT.mkdir(parents=True, exist_ok=True)
10
11
12 def api_get(method, params=None):
13     for _ in range(3):
14         try:
15             data = requests.get(f"{API}/{method}", params=params, timeout=30).json()
16             return data["result"] if data.get("status") == "OK" else None
17         except Exception:
18             time.sleep(5)
19     return None
20
21
22 def main():
23     contests = [c for c in api_get("contest.list", {"gym": "false"}) if c["phase"] == "FINISHED"
24                 ]
25     for c in sorted(contests, key=lambda c: c["id"]):
26         path = OUT / f"{c['id']}.json"
27         if path.exists():
28             continue
29         rc = api_get("contest.ratingChanges", {"contestId": c["id"]}) or []
30         path.write_text(json.dumps(rc))
31         time.sleep(2)
32
33 if __name__ == "__main__":
34     main()

```

build_features.py

```

1 import csv
2 import json
3 import statistics
4 from pathlib import Path
5
6 ROOT = Path(__file__).resolve().parent.parent
7 RAW = ROOT / "data" / "rating_changes"
8 OUT = ROOT / "data" / "features.csv"
9
10 FIELDS = [
11     "contest_id", "division", "handle", "time", "old_rating", "num_participants",
12     "field_avg_rating", "prior_contest_count", "prior_avg_rating_change",
13     "prior_rating_change_std", "prior_best_rank", "prior_avg_rank",
14     "days_since_last_contest", "rating_trend_last3", "rating_change",
15 ]
16
17
18 def division(contest_name):
19     n = contest_name.lower()
20     if "div. 1" in n and "div. 2" in n:
21         return "Div1+2"
22     for d in ("1", "2", "3", "4"):
23         if f"div. {d}" in n:
24             return f"Div{d}"
25     if "educational" in n:
26         return "Educational"

```

```

27     if "global" in n:
28         return "Global"
29     if "icpc" in n:
30         return "ICPC"
31     return "Other"
32
33
34 def main():
35     contests = []
36     for p in RAW.glob("*.json"):
37         rows = json.loads(p.read_text())
38         if rows:
39             contests.append((int(p.stem), rows))
40     contests.sort(key=lambda t: t[1][0]["ratingUpdateTimeSeconds"])
41
42     history = {}
43     with OUT.open("w", newline="") as f:
44         writer = csv.DictWriter(f, fieldnames=FIELDS)
45         writer.writeheader()
46         for contest_id, rows in contests:
47             div = division(rows[0]["contestName"])
48             field_avg = statistics.mean(r["oldRating"] for r in rows)
49             for r in rows:
50                 h = history.setdefault(r["handle"], [])
51                 if h:
52                     deltas = [new - old for _, old, new, _ in h]
53                     ranks = [rank for _, rank in h]
54                     prior = {
55                         "prior_contest_count": len(h),
56                         "prior_avg_rating_change": statistics.mean(deltas),
57                         "prior_rating_change_std": statistics.pstdev(deltas) if len(deltas) > 1
58                             else 0.0,
59                         "prior_best_rank": min(ranks),
60                         "prior_avg_rank": statistics.mean(ranks),
61                         "days_since_last_contest": (r["ratingUpdateTimeSeconds"] - h[-1][0]) /
62                             86400,
63                         "rating_trend_last3": sum(deltas[-3:]),
64                     }
65                 else:
66                     prior = {k: "" for k in FIELDS if k.startswith(("prior_", "days_", "
67                         rating_trend"))}
68                     prior["prior_contest_count"] = 0
69                 writer.writerow({
70                     **prior,
71                     "contest_id": contest_id, "division": div, "handle": r["handle"],
72                     "time": r["ratingUpdateTimeSeconds"], "old_rating": r["oldRating"],
73                     "num_participants": len(rows), "field_avg_rating": round(field_avg, 1),
74                     "rating_change": r["newRating"] - r["oldRating"],
75                 })
76                 h.append((r["ratingUpdateTimeSeconds"], r["oldRating"], r["newRating"], r["rank"]
77                     )))
78
79 if __name__ == "__main__":
80     main()

```

train.py

```

1 import csv
2 from pathlib import Path
3

```

```

4 import numpy as np
5
6 SRC = Path(__file__).resolve().parent.parent / "data" / "features.csv"
7 DIVISIONS = ["Div1", "Div2", "Div3", "Div4", "Div1+2", "Global", "ICPC"]
8
9
10 def load():
11     with SRC.open(newline="") as f:
12         return list(csv.DictReader(f))
13
14
15 def build_matrix(rows):
16     n = len(rows)
17     get = lambda key: np.array([float(r[key] or 0) for r in rows])
18     old, field = get("old_rating"), get("field_avg_rating")
19     participants, prior_count = get("num_participants"), get("prior_contest_count")
20     y = get("rating_change")
21
22     is_debut = (prior_count == 0).astype(float)
23     div_dummies = np.array([[1.0 if r["division"] == d else 0.0 for d in DIVISIONS] for r in
24                             rows])
25
26     X = np.column_stack([
27         np.ones(n), old - field, field, np.log(participants), np.log1p(prior_count), is_debut,
28         get("prior_avg_rating_change"), get("prior_rating_change_std"), get("prior_best_rank"),
29         get("prior_avg_rank"), get("days_since_last_contest"), get("rating_trend_last3"),
30         div_dummies,
31     ])
32     return X, y
33
34 def chronological_split(contest_ids, train_frac=0.8, val_frac=0.1):
35     ids = np.asarray(contest_ids)
36     starts = np.flatnonzero(np.r_[True, ids[1:] != ids[:-1]])
37     n = len(ids)
38     train_end = int(starts[np.searchsorted(starts, int(train_frac * n))])
39     val_end = int(starts[np.searchsorted(starts, int((train_frac + val_frac) * n))])
40     return train_end, val_end
41
42 def main():
43     rows = load()
44     X, y = build_matrix(rows)
45     train_end, val_end = chronological_split([int(r["contest_id"]) for r in rows])
46     X_train, y_train = X[:train_end], y[:train_end]
47     X_val, y_val = X[train_end:val_end], y[train_end:val_end]
48
49     beta = np.linalg.solve(X_train.T @ X_train + 1e-6 * np.eye(X_train.shape[1]), X_train.T @
50                             y_train)
51
52     for label, Xs, ys in (("train", X_train, y_train), ("val", X_val, y_val)):
53         err = Xs @ beta - ys
54         print(f"{label}: MAE {np.abs(err).mean():.2f}, RMSE {np.sqrt((err ** 2).mean()):.2f}")
55
56 if __name__ == "__main__":
57     main()

```